

JavaMoney: An Emerging Feature for Monetary Functions in Banking and Finance

JSR 354, which is also termed as JavaMoney, provides a set of money and currency APIs for Java. These APIs provide support for standard ISO-4217 and custom currencies. They also help to bridge the gap in the representation of a monetary amount. JSR 354 provides an API for representing, transporting and performing comprehensive calculations with money and currency.



Money is a very common concept, used for most application implementations across different domains. But when it comes to Java based development, there is no dedicated support for currencies or monetary arithmetic libraries and APIs. The existing `java.util.Currency` class is strictly a structure used for representing ISO-4217 standard currencies, and doesn't provide flexible or standardised calculation and conversion facilities.

It was in Java 1.4 that `java.util.Currency` got introduced with a standard three-letter currency notation like USD or INR. Later, in the Java 7 upgrade, a support for three-letter numeric code representation like USD=B40 was introduced. Also, there is no standard value type to represent a monetary amount or currency arithmetic functions in the current version of Java. Either the user has to rely on some popular third party library (like Joda) or write customised code that may not be generic to global conventions.

JavaMoney supports currency arithmetic across different currencies, as well as foreign currency exchange.

Monetary values are a key feature of many applications, mainly in banking, financial services, insurance and other such similar domains.

What is money?

Money is a simple representation of an amount (value) and the currency (unit). For example, with US\$ 100, '100' is the amount and US\$ is the unit/currency representing this amount.

How do we represent money so far

Till now, application development in Java either used a currency interface, or used 'double' or 'BigDecimal' for money representation. But this has a side effect when the floating point is a binary approximation of a decimal. Though the approximation is small decimal, this has its own meaning when it comes to the calculation for money. This kind of convention is good when the logic handles the value by rounding the floating point and doesn't consider the smallest fractional values.

Though `BigDecimal` doesn't cause such a floating point approximation, it is slow in calculation due to internal conversion and often requires rounding mode to handle the values represented.

Sometimes, we also use 'long' to represent money, which is a much better choice than the above two representations, but since these representations don't associate a unit (currency) with them, they can easily confuse unit values—for example, dollars or euros with cent values.

Using third party libraries

We now have many sophisticated libraries from third parties that can help in bridging the gap left by money representation in our application programs. These also help in extending the features to enable custom development.

One of the popular representations is `JodaMoney`, which is a simple wrapper on `BigDecimal` and provides conventional features for money related arithmetic functions.

Another representation includes `JScience`, which focuses more on units, and `Eclipse UoMo`, a proposed open source initiative under the Eclipse Technology Project. It is intended to support unit and measurement APIs (by implementing APIs